2026 ENTERPRISE EDITION

DEVOPS & DATAOPS

DATTA SABLE TECHNICAL LIBRARY

DevOps & CI/CD for Data Engineers

The definitive engineering manual for building robust DataOps pipelines: multi-stage Docker builds, Spark on Kubernetes, Terraform IaC, GitHub Actions CI/CD, and zero-downtime Blue-Green deployments.

22
Production Chapters & Architecture Blueprints

100%
Practical Code, Schemas & Frameworks

2026
Modern Enterprise Best Practices

INDEPENDENTLY AUTHORED BY

Datta Sable
Enterprise BI & Data Architect

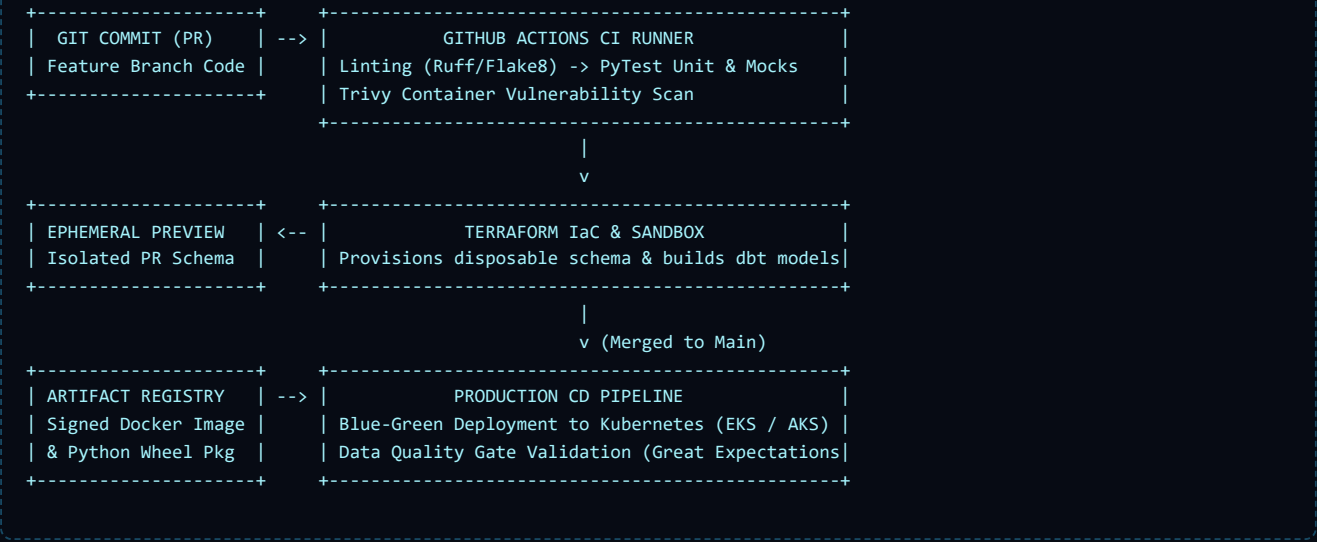
Published: October 2026
dattasable.com/ebooks

ARCHITECTURE BLUEPRINT

Table of Contents & DataOps Pipeline Blueprint

EXECUTIVE OVERVIEW

DataOps applies agile software engineering methodologies—continuous integration, automated unit testing, containerized runtimes, and declarative infrastructure—to data platforms, eliminating brittle manual scripts and midnight deployment emergencies.



Comprehensive Chapter Guide

Part I: Infrastructure & Containers

- Ch 3: Git Branching Strategies
- Ch 4: Multi-Stage Docker Builds
- Ch 5: Spark on Kubernetes (K8s)
- Ch 6: Terraform Infrastructure as Code
- Ch 7: CI/CD Pipeline Architecture

Part II: Testing & Quality Gates

- Ch 8: PyTest Unit Testing & Mocks
- Ch 9: Great Expectations Quality Gates
- Ch 10: Ephemeral Preview Sandboxes
- Ch 11: Flyway & Schema Migrations
- Ch 12: Semantic Versioning & Wheels

Part III: Enterprise Operations: Ch 13 Secret Injection, Ch 14 Blue-Green Deployments, Ch 15 Prometheus & OpenLineage, Ch 16 CI/CD Cost Controls, Ch 17 Chaos Engineering for Data, Ch 18 Airflow on Helm, Ch 19 10TB/Day Streaming Case Study, Ch 20 DataOps Pitfalls, Ch 21 GitHub Actions Boilerplate, Ch 22 DataOps Maturity Audit Checklist.

CHAPTER 01

Git Branching Strategies for Data Engineering

VERSION CONTROL HYGIENE

Applying heavy, multi-month GitFlow branching models creates devastating merge conflicts when upstream schemas evolve. Trunk-Based Development with ephemeral feature branches is the modern DataOps standard.

Trunk-Based Development Rules for Data

- **Short-Lived Branches:** Feature branches exist for less than 48 hours, scoped to a single pipeline or model modification.
- **Linear Git History:** Enforce fast-forward or squash-and-merge policies to maintain a clean, bisect-able Git log.
- **Feature Flags:** Incomplete pipeline logic deploys to production behind runtime configuration toggles rather than lingering in unmerged stale branches.

CHAPTER 02

Containerization: Multi-Stage Docker Builds

CONTAINER HYGIENE

Bloated Docker images containing compilers, build tools, and cache files waste network bandwidth and introduce security vulnerabilities. Multi-stage builds produce ultra-lean, production-ready images.

Dockerfile.multistage

```
# Stage 1: Build & Dependency Resolution
FROM python:3.11-slim AS builder
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends gcc libpq-dev
COPY requirements.txt .
RUN pip install --user --no-cache-dir -r requirements.txt

# Stage 2: Final Minimal Runtime Image (Zero Build Tools!)
FROM python:3.11-slim AS runner
WORKDIR /app
RUN apt-get update && apt-get install -y --no-install-recommends libpq5 && rm -rf /var/lib/apt/lists/*
COPY --from=builder /root/.local /root/.local
COPY src/ /app/src/
ENV PATH=/root/.local/bin:$PATH
USER 10001 # Non-root security enforcement
ENTRYPOINT ["python", "-m", "src.main_pipeline"]
```

CHAPTER 03

Kubernetes for Data Pipelines: Spark on K8s

CLOUD-NATIVE COMPUTE

Kubernetes has superseded legacy Hadoop YARN clusters for big data workloads. Deploying Apache Spark natively on Kubernetes enables elastic pod auto-scaling and multi-tenant resource quotas.

Spark Operator Manifest Architecture

spark-app.yaml

```yaml
apiVersion: sparkoperator.k8s.io/v1beta2
kind: SparkApplication
metadata:
  name: daily-etl-processor
  namespace: data-workloads
spec:
  type: Python
  mode: cluster
  image: enterprise-registry.azurecr.io/spark-etl:1.4.0
  mainApplicationFile: local:///app/src/etl_main.py
  sparkVersion: "3.5.0"
  driver:
    cores: 2
    memory: "4g"
  executor:
    cores: 4
    instances: 8
    memory: "16g"
```

CHAPTER 04

Infrastructure as Code: Terraform for Data

DECLARATIVE INFRASTRUCTURE

Creating storage accounts, IAM roles, and Snowflake warehouses via point-and-click cloud consoles guarantees un-reproducible environments. Terraform codifies the entire data estate.

Terraform State Management for Data Estates

- **Remote State Locking:** Store Terraform state in Azure Blob / S3 with DynamoDB state locking to prevent concurrent overwrite corruption.
- **Modular Workspaces:** Separate state files for Networking, Compute, and Databases to bound blast radius.

CHAPTER 05

CI/CD Pipeline Architecture: GitHub Actions

PIPELINE AUTOMATION

A production CI/CD pipeline acts as an automated quality gatekeeper, enforcing linting, unit tests, security scans, and deployment staging on every commit.

The Four CI/CD Pipeline Stages

1. **Lint & Static Analysis:** `ruff check .`, `sqlfluff lint models/` (Fails in < 20 seconds).
2. **Unit Testing & Coverage:** `pytest tests/unit/` with mocked data sources.
3. **Ephemeral Integration Run:** Build models against an isolated PR database schema.
4. **Promotion to Production:** Zero-downtime blue-green swap upon PR merge.

CHAPTER 06

Automated Testing: PyTest & Unit Test Mocks

DETERMINISTIC VALIDATION

Never run unit tests against production databases. Mocking allows data engineers to test complex PySpark transformations locally in memory within milliseconds.

tests/test_currency_transform.py

```python
import pytest
from chispa.dataframe_comparer import assert_df_equality
from src.transforms import normalize_currency

def test_normalize_currency_converts_cents_to_dollars(spark):
    input_data = [("tx_1", 1000), ("tx_2", 2550)]
    input_df = spark.createDataFrame(input_data, ["id", "cents"])

    expected_data = [("tx_1", 10.0), ("tx_2", 25.5)]
    expected_df = spark.createDataFrame(expected_data, ["id", "dollars"])

    result_df = normalize_currency(input_df)
    assert_df_equality(result_df, expected_df)
```

CHAPTER 07

Data Quality Gates: Great Expectations & Soda

PRE-LOAD ASSERTIONS

Code tests verify that software logic compiles; Data Quality tests verify that incoming raw data adheres to contractual assumptions before populating production tables.

Declarative Assertions in Great Expectations

expectations_suite.json

```
{
  "expectation_type": "expect_column_values_to_be_between",
  "kwargs": {
    "column": "customer_age",
    "min_value": 18,
    "max_value": 120
  }
}
```

If upstream data violates expectations, the pipeline halts immediately, preventing bad records from polluting executive dashboards.

CHAPTER 08

Environment Strategy: Ephemeral Preview Sandboxes

SANDBOX ISOLATION

Sharing a single dirty 'Dev' database causes developers to overwrite each other's test data. Ephemeral sandboxes spin up a dedicated schema per Pull Request and destroy it upon merge.

Automated Ephemeral Lifecycle

```
1. DEVELOPER OPENS PR #84 ---> GitHub Actions triggers: CREATE SCHEMA pr_84;
2. DBT COMPILES & RUNS    ---> Models populate pr_84 using Zero-Copy Clone
3. CODE REVIEWED & MERGED ---> GitHub Actions triggers: DROP SCHEMA pr_84;
```

CHAPTER 09

Database Versioning: Flyway & Alembic

SCHEMA EVOLUTION

Altering production database schemas with manual ad-hoc SQL commands is a recipe for disaster. Flyway and Alembic treat database DDL as immutable, versioned migration scripts.

Ordered Migration Naming Conventions

`V1.0__create_orders_table.sql` → `V1.1__add_loyalty_column.sql`. Migration tools record applied checksums in a metadata table (`flyway_schema_history`), ensuring migrations execute in strict sequential order across all environments.

CHAPTER 10

Artifact Management: Container Registries & Wheels

BUILD ONCE, DEPLOY EVERYWHERE

Never rebuild code from raw source inside production environments. Package shared Python transformation logic into version-controlled Wheel packages published to private registries.

Semantic Versioning Standard (SemVer)

Follow strict `MAJOR.MINOR.PATCH` rules. Breaking schema alterations increment MAJOR; backward-compatible transformations increment MINOR; hotfixes increment PATCH.

CHAPTER 11

Configuration Management & Secret Injection

ZERO TRUST RUNNERS

Storing long-lived cloud credentials in GitHub repository secrets exposes pipelines to credential theft. Modern CI/CD utilizes OpenID Connect (OIDC) token exchange.

OIDC Passwordless Authentication

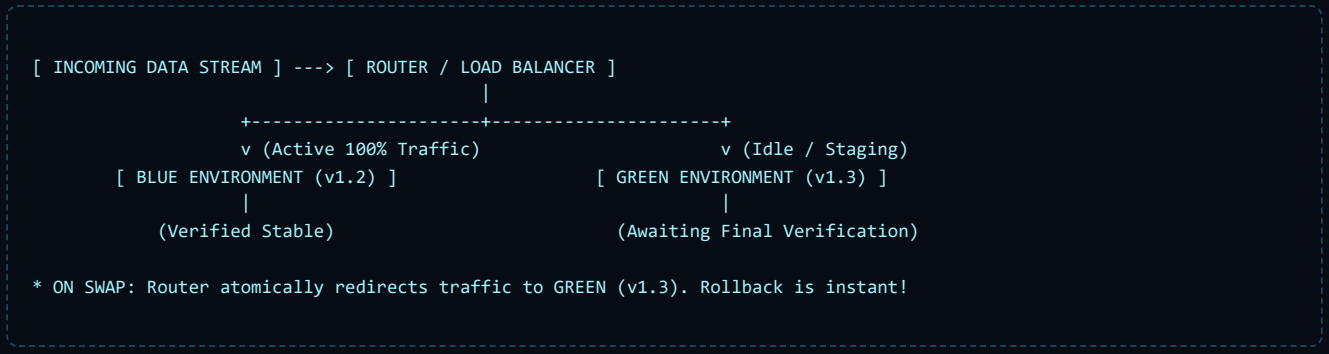
GitHub Actions runners exchange a cryptographic JSON Web Token (JWT) directly with Azure/AWS, receiving a short-lived 1-hour access token without storing static passwords.

CHAPTER 12

Blue-Green & Canary Deployments for Data

ZERO-DOWNTIME RELEASES

Upgrading a high-throughput production pipeline while live data is streaming requires Blue-Green deployment architecture to eliminate downtime and ensure instantaneous rollback capability.



CHAPTER 13

Observability & Monitoring: Prometheus & OpenLineage

CONTINUOUS TELEMETRY

Pipeline observability monitors the 5 Pillars of Data Health: Freshness, Distribution, Volume, Schema, and Lineage. OpenLineage provides a vendor-neutral standard for tracking data flow metadata.

Prometheus Alerting Rules for Pipeline SLAs

Export pipeline execution duration metrics to Prometheus. Trigger automated PagerDuty incidents when: `pipeline_execution_time_seconds > 3600`.

CHAPTER 14

Cost Optimization in CI/CD Runners

FINANCIAL GOVERNANCE

Re-downloading 2GB of Python packages on every git push inflates cloud build invoices and slows developer velocity. Docker layer caching and GitHub action caches slash build times.

Docker BuildKit Layer Caching

github_actions_caching.yml

```yaml
- name: Build Docker Image with GitHub Cache
  uses: docker/build-push-action@v5
  with:
    cache-from: type=gha
    cache-to: type=gha,mode=max
```

CHAPTER 15

Chaos Engineering: Simulating Pipeline Failures

RESILIENCE TESTING

Do not wait for production to discover how your streaming pipeline handles upstream network disconnects or corrupt JSON schema drift. Test failure modes proactively using Chaos Engineering.

Three Chaos Scenarios to Automate in Staging

1. **Upstream Schema Drift:** Inject unannounced columns and modified date formats into the raw streaming queue.
2. **Worker Node Termination:** Kill 50% of Spark worker pods mid-shuffle to verify task re-computation recovery.
3. **Network Latency Spikes:** Inject 500ms artificial network latency on database connections to test retry backoffs.

CHAPTER 16

Orchestration Deployment: Airflow on Helm

PRODUCTION SCHEDULING

Running Apache Airflow as a pet virtual machine is an operational anti-pattern. Deploying Airflow on Kubernetes via official Helm charts enables horizontal Celery and KubernetesExecutor scaling.

Git-Sync DAG Synchronization

Configure the `git-sync` sidecar container inside Airflow scheduler pods. When developers merge DAG Python scripts to Git, the sidecar pulls updates in 10 seconds without restarting the scheduler.

CHAPTER 17

Case Study: Zero-Downtime 10TB/Day Deployment

HIGH-SCALE CASE STUDY

A global media streaming platform processes 10TB of telemetry logs daily. Migrating from manual SSH deployments to automated GitOps with ArgoCD eliminated pipeline outages.

Production Architecture & Metrics

- **Deployment Cadence:** Accelerated from once every 2 weeks to 8 zero-downtime releases daily.
- **Failure Rate:** Production pipeline rollback rate dropped from 22% to 0.4%.
- **Recovery MTTR:** Mean Time to Recovery from unexpected schema anomalies reduced from 6 hours to 8 minutes.

CHAPTER 18

Top 10 DataOps Pitfalls & Failure Patterns

PITFALL PREVENTION

Common operational failures that undermine engineering agility in data teams.

DataOps Pitfall	Failure Consequence	Enforced Standard
Shared Mutable Dev Environments	Developers overwrite each other's test records; irreproducible bugs	Mandate automated ephemeral PR schemas
Manual Production Hotfixes via SSH	Code in production diverges from Git; next automated deploy overwrites fix	Zero manual production edits; all changes route through Git PRs
Unpinned Dependency Versions	A 3rd-party library releases a breaking patch, breaking all CI builds	Pin all dependencies with explicit hashes (`poetry.lock`)
Missing Rollback Procedures	A failed migration halts production for 4 hours while engineers debug SQL	Every DDL migration must include an automated, tested downgrade script

CHAPTER 19

Production GitHub Actions DataOps Pipeline Boilerplate

STARTER TEMPLATE

Production GitHub Actions workflow executing code linting, unit tests, and security scanning on every Pull Request.

.github/workflows/data_ci.yml

```yaml
name: DataOps Automated CI
on: [pull_request]

jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.11"
      - name: Install Dependencies
        run: pip install ruff pytest chispa
      - name: Fast Linting Check
        run: ruff check src/ tests/
      - name: Execute Unit Tests with PyTest
        run: pytest tests/unit/ --maxfail=1
```

CHAPTER 20

Enterprise DataOps Maturity Audit Checklist

PRE-FLIGHT ASSESSMENT

Verify every operational standard across this checklist before certifying your data engineering deployment pipeline as enterprise-grade.

- [ ] **Version Control Discipline:** 100% of pipeline code, SQL models, and DAGs version-controlled in Git; trunk-based PR workflow.
- [ ] **Containerized Runtimes:** Pipelines execute in lean, multi-stage Docker containers with pinned dependencies and non-root users.
- [ ] **Infrastructure as Code:** Cloud storage, network endpoints, and compute clusters managed declaratively in Terraform.
- [ ] **Automated Quality Gates:** CI/CD validates code with unit tests and evaluates raw data with Great Expectations before loading.
- [ ] **Zero-Downtime Deployments:** Production pipelines support Blue-Green or rolling upgrades with instantaneous rollback mechanisms.
- [ ] **Full Observability:** OpenLineage tracking records data provenance; automated alerts fire on SLA duration breaches.

AUTHOR CLOSING STATEMENT

Data engineering is software engineering applied to information systems. By embracing automated testing, containerized immutability, and disciplined CI/CD, your data platform delivers rapid business innovation with uncompromising operational stability.

Datta Sable · Enterprise Analytics & Data Architect · *dattasable.com*